

* Module Wrapper :-

— Every module in node.js gets wrapped in an IIFE before being loaded.

— IIFE helps keep top-level variables scoped to the module rather than the global object/variable.

— The IIFE That wraps every module contains 5 [Five] parameters which are pretty important for the functioning of a module.

iife1.js // Simple Previous Version
// Without parameter.

```
(function() {
  const superhero = "Batman";
  console.log(superhero);
})();

(function() {
  const superhero = "Superman";
  console.log(superhero);
})();
```

iife2.js // Parameter Version

// With parameter - In Passing & In Calling

```
(function(msg) {
  const superhero = "Batman";
  console.log(msg, superhero);
})("Hello"); // msg - Passing Argument

(function(msg) {
  const superhero = "Superman";
  console.log(msg, superhero);
})("Hi");
```

```
% node iife1
Batman
Superman
%
```

```
% node iife2
Hello Batman
Hi Superman
%
```

— This is how we specify parameters & pass arguments to an immediately invoked function expression.

* Understanding module How it wrap

```
(function(exports, require, module, __filename, __dirname) {
  // module code
})
```

Final code with parameters.

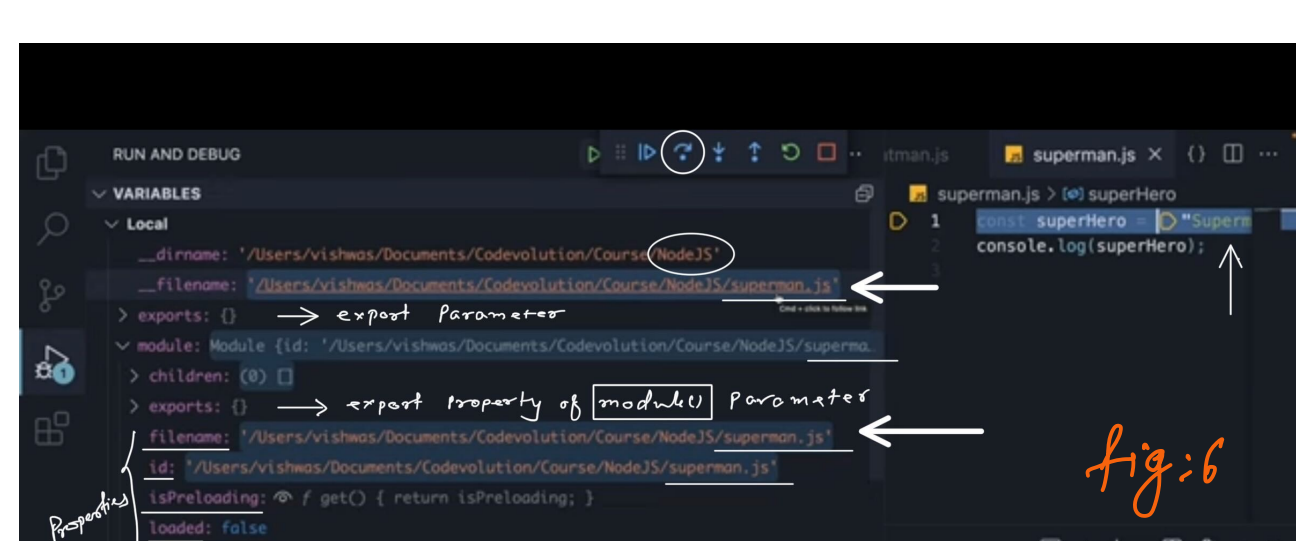
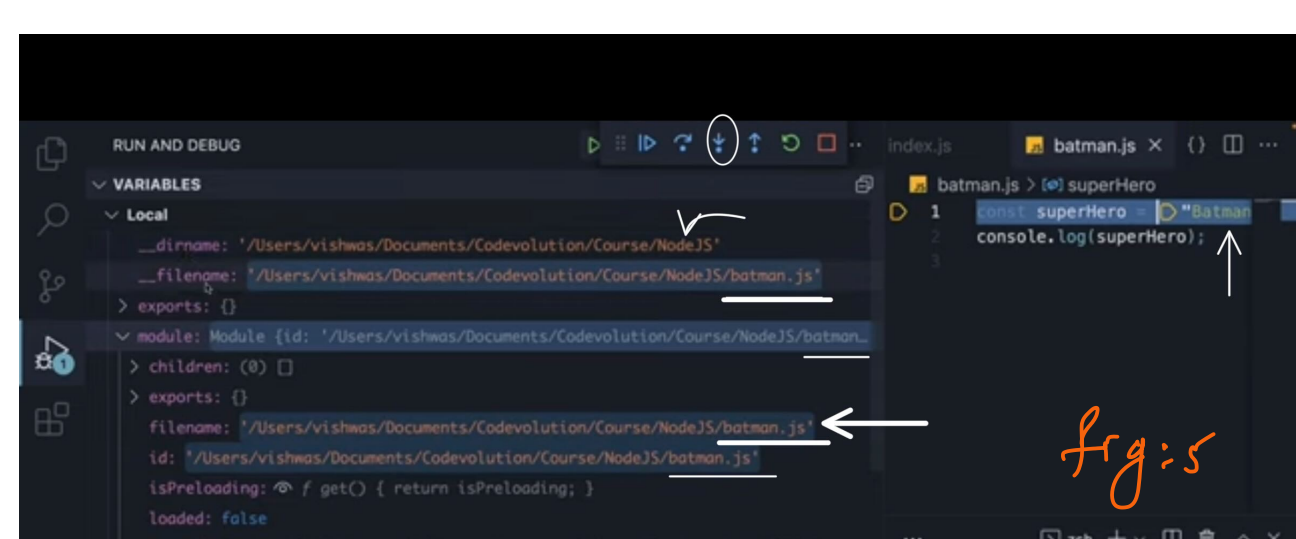
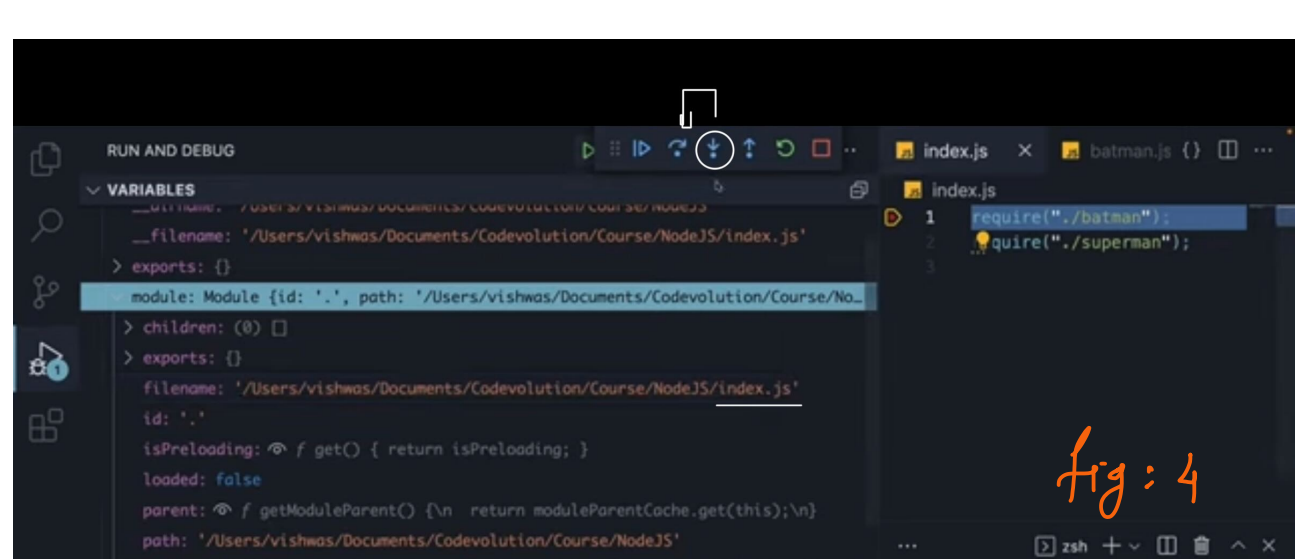
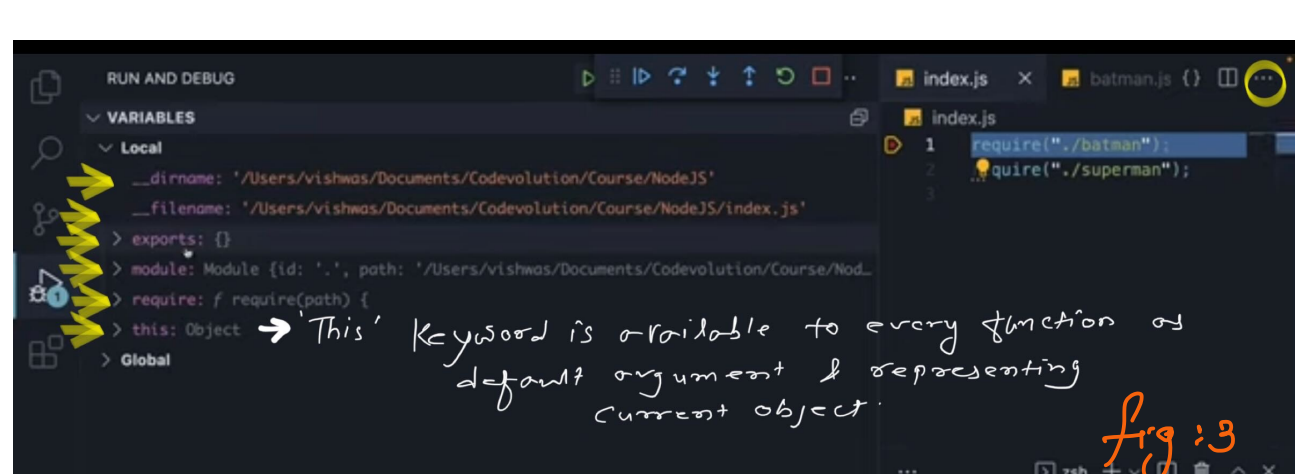
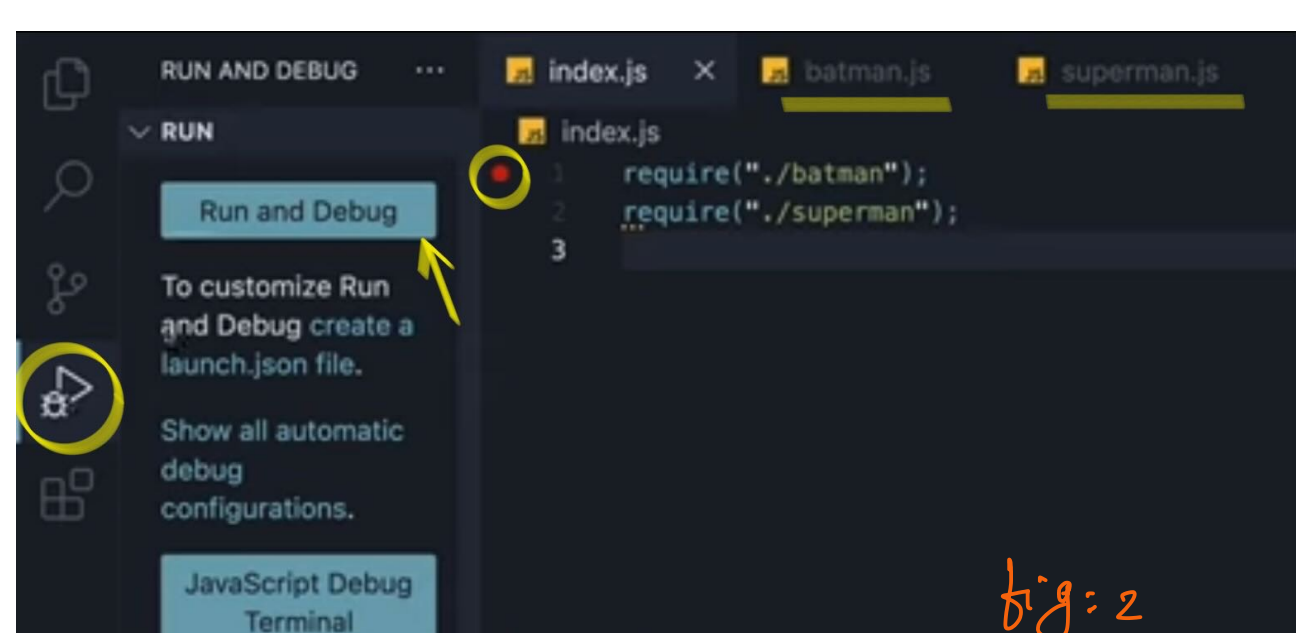
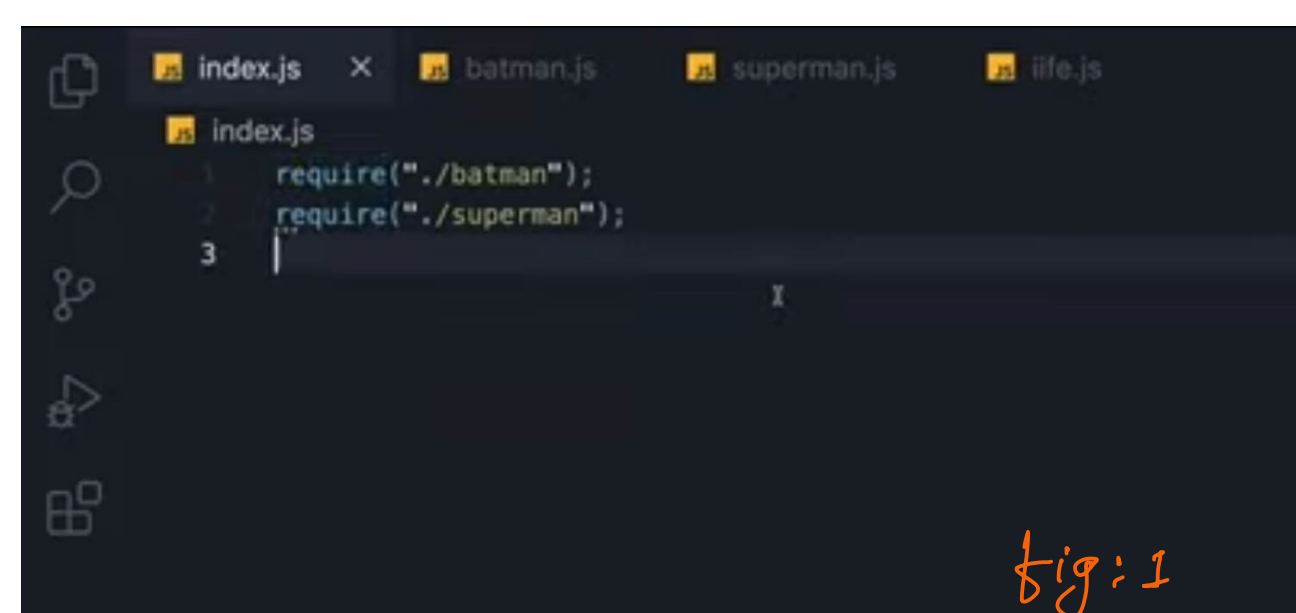
— There are five in total.

— By wrapping each module code in each iife with 5 [five] parameters,

— node.js provides few global looking variables that are actually specific to the module.

— Let understand with previous example

How node treats the module as wrapper?



— Total 3 files.
"batman.js"
"superman.js" &
"index.js" — Total 3 files.

In "index.js" we call 2 files using require() & call as shown in fig:1

— By executing "index.js" in "Run & Debug" mode --- & using tool bar option "Step in" & "Step over" as shown in fig:2

— We can see all 5 parameters on left side pane ---

— Examine One by one

— We can see initially

-- dirname & __filename Variables contain current dir path & current file respectively - "index.js"

— As we "Step in" - 1st require() & executes of Node in Variables - data changes

-- filename now stores "batman.js" instead "index.js"

— As we step-over again we can see that our second require() & executes of Node data of

-- filename becomes current file "superman.js"

— Carefully examine content of each parameter.

— Also understand the difference between "exports" parameter & "export" property of "module" parameter.

* We will understand "export parameter" in more detail in next coming lectures.

-- dirname : } both are convenience variables available for use within the module.
-- filename :